Matemática para Aprendizaje Automático

Notas de Clase — Factorizaciones de Matrices

Universidad Católica del Uruguay • 2026

> ***Referencias principales:*** *Deisenroth, Faisal & Ong, Mathematics for Machine Learning, sección 4.3 (Cholesky); Strang, Introduction to Linear Algebra, secciones 2.6 (LU) y 4.4 (QR).* ***Referencias complementarias:*** *Aggarwal, Linear Algebra and Optimization for Machine Learning, secciones 2.5, 2.7, 3.3.9; Trefethen & Bau, Numerical Linear Algebra, Lectures 7–8 (QR), 20–21 (LU) y 23 (Cholesky); Golub & Van Loan, Matrix Computations, secciones 3.2 (LU), 4.2 (Cholesky) y 5.2 (QR).*

1. Factorización de matrices

1.1 Qué es una factorización

Una **factorización** de una matriz A es una expresión de A como producto de dos o más matrices con propiedades específicas:

$$A = B_1 B_2 \cdots B_k.$$

La utilidad de una factorización depende del tipo de matrices B_i que intervienen: matrices triangulares, ortogonales, diagonales, de permutación, etc. Cada elección de estructura revela un aspecto distinto de A y habilita algoritmos más eficientes o numéricamente más estables para problemas concretos.

Un ejemplo que ya conocemos es la **diagonalización** o descomposición en valores y vectores propios:

$$A = PDP^{-1},$$

donde D es diagonal (con los valores propios de A) y P es invertible (con los vectores propios de A como columnas).

1.2 Factorizaciones que veremos

En estas notas estudiamos tres factorizaciones clásicas del álgebra lineal numérica:

Factorización	Expresión	Condición sobre A	Costo (matriz $n \times n$)
LU	$A = LU$	cuadrada (con pivoteo, cualquier matriz no singular)	$\frac{2}{3}n^3$
Cholesky	$A = LL^\top$	simétrica definida positiva	$\frac{1}{3}n^3$
QR	$A = QR$	rango completo por columnas	$\approx 2mn^2$ (matriz $m \times n$)

En $A = LU$, L es triangular inferior y U triangular superior. En Cholesky, L es triangular inferior. En QR, Q tiene columnas ortonormales y R es triangular superior.

En la siguiente unidad veremos la **descomposición en valores singulares (SVD)**, que generaliza la diagonalización a matrices rectangulares:

$$A = U\Sigma V^\top,$$

con U, V ortogonales y Σ diagonal con los valores singulares de A.

Existen muchas otras factorizaciones matriciales que no veremos en el curso. A modo de ejemplo:

- **Descomposición de Schur:** $A = QTQ^\top$ con Q ortogonal y T triangular superior.
- **Factorización matricial no negativa (NMF):** $A = WH$ con $W, H \geq 0$ (útil en análisis de datos textuales, descomposicion por partes y sistemas de recomendación).
- **Descomposición polar:** $A = UP$ con U ortogonal y P simétrica semidefinida positiva.

1.3 Para qué factorizar matrices

Factorizar una matriz es atractivo por cuatro motivos recurrentes:

1. **Eficiencia:** resolver un problema sobre las matrices B_i suele ser más barato que resolverlo sobre A. Por ejemplo, resolver $Tx = b$ con T triangular cuesta $O(n^2)$ mientras que resolver $Ax = b$ sin estructura cuesta $O(n^3)$.
2. **Estabilidad numérica:** ciertas factorizaciones (Cholesky, QR con Householder) producen algoritmos numéricamente estables para problemas comunes como la resolución de sistemas lineales o mínimos cuadrados.
3. **Análisis estructural:** los factores revelan propiedades de A: determinante, valores propios, número de condición, rango, signo.
4. **Compresión y aproximación:** truncar una factorización produce aproximaciones de baja complejidad. La SVD, por ejemplo, permite obtener la mejor aproximación de rango k de una matriz en el sentido de la norma de Frobenius (ver notas de SVD).

---

2. Descomposición LU

> ***Lecturas sugeridas:*** *Strang, Introduction to Linear Algebra, sección 2.6.*

2.1 Definición

La descomposición LU expresa una matriz cuadrada $A \in \mathbb{R}^{n\times n}$ como $A = LU$, donde:

- L es triangular inferior **unitaria** (es decir, con 1's en la diagonal),
- U es triangular superior.

> ***Nota.*** *La convención de que L sea la unitaria es arbitraria — también podría pedirse que sea U la que tenga diagonal unitaria. Es necesario fijar una convención para garantizar la unicidad de la factorización (ver §2.7).*

Para una matriz 3×3:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}.$$

La descomposición LU es la versión matricial del método de **eliminación gaussiana**: las filas de U son las filas que quedan al escalonar A, y los coeficientes ℓ_{ij} son los multiplicadores que se usaron en cada paso de eliminación.

2.2 Algoritmo y costo computacional

Existen una variedad de algoritmos, equivalentes, para obtener la descomposición LU. El siguiente procedimiento obtiene L y U *in situ* (sobrescribiendo A, lo que ahorra memoria aunque no cómputo):

1. Inicializar $L = I$ y $U = A$.
2. Para cada columna $k = 1, 2, \ldots, n-1$:
   - Para cada fila $i = k+1, \ldots, n$ bajo la diagonal:
     - Calcular el multiplicador $\ell_{ik} = u_{ik}/u_{kk}$.
     - Guardarlo en L: $L_{ik} \leftarrow \ell_{ik}$.
     - Actualizar la fila i de U restándole ℓ_{ik} veces la fila k: $u_{ij} \leftarrow u_{ij} - \ell_{ik}\, u_{kj}$ para $j = k, k+1, \ldots, n$.

En el paso k-ésimo se realizan $n-k$ actualizaciones de fila, cada una con una división y $2(n-k)$ operaciones aritméticas. El costo total es:

$$\sum_{k=1}^{n-1} (n-k)\big(2(n-k)+1\big) = \tfrac{2}{3}n^3 + O(n^2).$$

Es decir, aproximadamente $\frac{2}{3}n^3$ operaciones, el mismo costo que la eliminación gaussiana.

2.3 Ejemplo: matriz 3×3

Consideremos $A = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{bmatrix}$. Inicializamos $L = I$ y $U = A$.

Paso 1 — eliminar bajo la diagonal en la columna 1.

- $\ell_{21} = 4/2 = 2$ y $\ell_{31} = 8/2 = 4$.
- Actualización: $r_i \leftarrow r_i - \ell_{i1}\, r_1$ para $i = 2, 3$.

$$U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 3 & 5 \end{bmatrix}.$$

Paso 2 — eliminar bajo la diagonal en la columna 2.

- $\ell_{32} = 3/1 = 3$.
- Actualización: $r_3 \leftarrow r_3 - 3\, r_2$.

$$U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix}.$$

Resultado final:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix}, \qquad U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix}.$$

Implementación en Numpy

El siguiente código ejemplifica el cálculo de la descomposición LU y su verificación. Numpy no expone la función de descomposición LU, pero se puede usar `scipy.linalg.lu` para obtener P, L y U con pivoteo parcial.

```
import numpy as np
# NumPy no tiene una función que expone la descomposición LU (aunque la utiliza cuando hacemos np.
# (scipy siempre devuelve la factorización con pivoteo parcial: PA = LU.)
from scipy.linalg import lu

A = np.array([[2, 1, 1],
              [4, 3, 3],
              [8, 7, 9]], dtype=float)

P, L, U = lu(A)
print(L)
print(U)
print(np.allclose(P @ L @ U, A))   # True
# Notar que la convención de Scipy para la descomposición PLU es diferente a la nuestra: en lugar
```

2.4 Pivoteo

Cuando u_{kk} es cero el multiplicador ℓ_{ik} no está definido; cuando u_{kk} es pequeño respecto al resto de la columna, los multiplicadores resultantes son grandes y amplifican errores de redondeo. Para evitar ambos problemas se usa **pivoteo**, que intercambia filas (y eventualmente columnas) antes de cada eliminación.

Matrices de permutación

Una **matriz de permutación** $P \in \mathbb{R}^{n\times n}$ es una matriz que se obtiene permutando las filas de la identidad. Equivalentemente, P tiene exactamente un 1 por fila y por columna, y ceros en el resto. Por ejemplo:

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

lleva la fila 1 a la posición 2, la fila 2 a la posición 3 y la fila 3 a la posición 1. Multiplicar PA por izquierda aplica esa permutación a las filas de A; multiplicar AP por derecha la aplica a las columnas.

Propiedad. Toda matriz de permutación es **ortogonal**: sus columnas son una permutación de los vectores canónicos $e_1, \ldots, e_n$, que forman una base ortonormal. Por lo tanto, $P^\top P = I$ y $P^{-1} = P^\top$.

Pivoteo parcial: $PA = LU$

En cada paso k, antes de eliminar, intercambiamos la fila k con aquella fila $i \geq k$ que tenga el mayor valor absoluto en la columna k (el "pivote"). La matriz P registra la composición de todos esos intercambios; el resultado es una factorización

$$PA = LU$$

que siempre existe si A es no singular.

Pivoteo completo: $PAQ = LU$

Se permiten además permutaciones de columnas (Q), eligiendo como pivote el elemento de mayor valor absoluto en toda la submatriz restante. Es más estable pero también más costoso, y rara vez se usa en la práctica.

Ejemplo de pivoteo parcial

Consideremos $A = \begin{bmatrix} 2 & 4 & -2 \\ 1 & 3 & 1 \\ 4 & 9 & -3 \end{bmatrix}$.

Paso 1. El mayor valor absoluto de la primera columna es 4, en la fila 3. Intercambiamos filas 1 y 3. La permutación parcial es $P_1 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$ y

$$P_1 A = \begin{bmatrix} 4 & 9 & -3 \\ 1 & 3 & 1 \\ 2 & 4 & -2 \end{bmatrix}.$$

Multiplicadores: $\ell_{21} = 1/4$ y $\ell_{31} = 2/4 = 1/2$. Tras eliminar:

$$\begin{bmatrix} 4 & 9 & -3 \\ 0 & 3/4 & 7/4 \\ 0 & -1/2 & -1/2 \end{bmatrix}.$$

Paso 2. El mayor valor absoluto en la columna 2 bajo la diagonal es $3/4$, ya en la fila 2. No hay intercambio ($P_2 = I$).

Multiplicador: $\ell_{32} = (-1/2)/(3/4) = -2/3$. Tras eliminar:

$$U = \begin{bmatrix} 4 & 9 & -3 \\ 0 & 3/4 & 7/4 \\ 0 & 0 & 2/3 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 1/4 & 1 & 0 \\ 1/2 & -2/3 & 1 \end{bmatrix}, \quad P = P_1.$$

Se verifica $PA = LU$.

2.5 LU para matrices rectangulares

La descomposición LU se puede extender a matrices rectangulares $A \in \mathbb{R}^{m \times n}$. Hay dos casos según las dimensiones.

Matriz "ancha" ($m \leq n$). L es cuadrada $m \times m$ triangular inferior unitaria, y U es $m \times n$ trapezoidal superior ($U_{ij} = 0$ para $i > j$). Por ejemplo, para $A \in \mathbb{R}^{2 \times 3}$:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 \\ \ell_{21} & 1 \end{bmatrix}}_{L\ (2\times 2)} \underbrace{\begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \end{bmatrix}}_{U\ (2\times 3)}.$$

Numéricamente, para $A = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 3 & 5 \end{bmatrix}$: $\ell_{21} = 1/2$, por lo que

$$L = \begin{bmatrix} 1 & 0 \\ 1/2 & 1 \end{bmatrix}, \qquad U = \begin{bmatrix} 2 & 4 & 6 \\ 0 & 1 & 2 \end{bmatrix}.$$

Matriz "alta" ($m \geq n$). L es $m \times n$ triangular inferior ("trapezoidal inferior") con diagonal unitaria, y U es cuadrada $n \times n$ triangular superior. Por ejemplo, para $A \in \mathbb{R}^{3\times 2}$:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 \\ \ell_{21} & 1 \\ \ell_{31} & \ell_{32} \end{bmatrix}}_{L\ (3\times 2)} \underbrace{\begin{bmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{bmatrix}}_{U\ (2\times 2)}.$$

En ambos casos puede combinarse con pivoteo parcial ($PA = LU$).

2.6 Aplicaciones

La descomposición LU ofrece una forma eficiente de resolver varios problemas matriciales. Dos destacan:

- resolución de sistemas lineales,
- cálculo del determinante.

2.6.1 Resolución de sistemas lineales

Dado $Ax = b$, si $A = LU$ el sistema se resuelve en dos pasos:

1. **Sustitución hacia adelante:** resolver $Ly = b$ (costo $O(n^2)$).
2. **Sustitución hacia atrás:** resolver $Ux = y$ (costo $O(n^2)$).

El costo total es $\frac{2}{3}n^3 + 2n^2$: la factorización domina. La ventaja aparece cuando hay que resolver k sistemas con la misma A y distintos b — los cuales pueden resolverse reutilizando L y U:

$$\text{costo total} \approx \tfrac{2}{3}n^3 + 2kn^2.$$

Mucho más barato que resolver cada sistema desde cero por eliminación gaussiana ($\frac{2}{3}kn^3$).

Con pivoteo. Si se usó pivoteo parcial ($PA = LU$), multiplicando $Ax = b$ por P a izquierda se obtiene $PAx = Pb$, es decir $LUx = Pb$. El procedimiento se ajusta en un solo paso: permutar b y luego sustituir como antes.

1. calcular $b' = Pb$,
2. sustitución hacia adelante: $Ly = b'$,
3. sustitución hacia atrás: $Ux = y$.

Permutar b cuesta $O(n)$ y no afecta el costo asintótico.

2.6.2 Cálculo del determinante

Usando la propiedad:

$$\det(AB) = \det(A) \cdot \det(B),$$

cualquier factorización $A = B_1 B_2 \cdots B_k$ permite calcular $\det(A)$ a partir de los determinantes de los factores. En particular, con $A = LU$ como $\det(L) = 1$ (triangular con diagonal unitaria) y $\det(U) = \prod_i u_{ii}$, se tiene

$$\det(A) = \prod_{i=1}^{n} u_{ii}.$$

Si hubo pivoteo ($PA = LU$), entonces $\det(A) = (-1)^s \prod_i u_{ii}$, donde s es el número de intercambios de fila. El determinante se obtiene al costo de la factorización LU, $\frac{2}{3}n^3$, muy por debajo de los $O(n!)$ de la definición por cofactores.

2.7 Existencia y unicidad de la factorización LU

Si A es no singular y admite descomposición LU sin pivoteo (es decir, el algoritmo de la sección 2.2 no encuentra ningún pivote nulo), entonces dicha descomposición es **única**. La unicidad depende de la convención de que L sea unitaria; también podría pedirse que sea U la que tenga diagonal unitaria, y entonces sería única esa otra descomposición.

Sin embargo, no toda matriz admite descomposición LU sin pivoteo. Un ejemplo inmediato es $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$: el primer pivote es 0 y el algoritmo falla. Con pivoteo parcial, en cambio, siempre se obtiene $PA = LU$ si A es no singular.

Condición suficiente para LU sin pivoteo: que todos los **menores principales** de A (los determinantes de las submatrices superior-izquierdas $A_{1:k,\,1:k}$, para $k = 1, \ldots, n$) sean no nulos.

2.8 Ejercicios sugeridos

1. Calcular la factorización LU de $A = \begin{bmatrix} 3 & 1 & 2 \\ 6 & 3 & 4 \\ 3 & 1 & 5 \end{bmatrix}$ sin pivoteo, y verificar $LU = A$.
2. Repetir el ejercicio anterior sobre $A = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & 3 \end{bmatrix}$ con pivoteo parcial; escribir explícitamente P.
3. Usar la factorización LU del primer ejercicio para calcular $\det(A)$.

---

3. Descomposición de Cholesky

> ***Lecturas sugeridas:*** *Deisenroth, Faisal & Ong, Mathematics for Machine Learning, sección 4.3.*

3.1 Definición

Para una matriz **simétrica definida positiva** (SDP) $A \in \mathbb{R}^{n\times n}$, la descomposición de Cholesky es

$$A = LL^\top,$$

donde L es triangular inferior con diagonal **estrictamente positiva**. Puede pensarse como una especie de "raíz cuadrada matricial" de A.

Desarrollando el producto para $n = 3$:

$$\begin{bmatrix} \ell_{11} & 0 & 0 \\ \ell_{21} & \ell_{22} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} \end{bmatrix} \begin{bmatrix} \ell_{11} & \ell_{21} & \ell_{31} \\ 0 & \ell_{22} & \ell_{32} \\ 0 & 0 & \ell_{33} \end{bmatrix} = \begin{bmatrix} \ell_{11}^2 & \ell_{11}\ell_{21} & \ell_{11}\ell_{31} \\ \ell_{11}\ell_{21} & \ell_{21}^2 + \ell_{22}^2 & \ell_{21}\ell_{31} + \ell_{22}\ell_{32} \\ \ell_{11}\ell_{31} & \ell_{21}\ell_{31} + \ell_{22}\ell_{32} & \ell_{31}^2 + \ell_{32}^2 + \ell_{33}^2 \end{bmatrix}.$$

3.2 Algoritmo y costo computacional

Igualando entradas y despejando columna por columna se obtiene el siguiente algoritmo. Para $j = 1, 2, \ldots, n$:

$$\ell_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} \ell_{jk}^2}, \qquad \ell_{ij} = \frac{1}{\ell_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} \ell_{ik}\, \ell_{jk} \right) \quad (i > j).$$

El algoritmo consta de n pasos. En el paso j-ésimo se calcula un elemento diagonal (costo aprox. $2j - 1$) y $n - j$ elementos subdiagonales (costo aprox. $2j - 1$ cada uno). El costo total es:

$$\sum_{j=1}^{n} (2j - 1)(n - j + 1) \approx \tfrac{1}{3} n^3 + O(n^2).$$

Cholesky cuesta la mitad que LU y no requiere pivoteo para matrices SDP.

3.3 Ejemplo: matriz SDP 3×3

Consideremos $A = \begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix}$.

Columna 1:

- $\ell_{11} = \sqrt{4} = 2$,
- $\ell_{21} = 12/2 = 6$,
- $\ell_{31} = -16/2 = -8$.

Columna 2:

- $\ell_{22} = \sqrt{37 - 6^2} = \sqrt{1} = 1$,
- $\ell_{32} = \big(-43 - 6 \cdot (-8)\big)/1 = 5$.

Columna 3:

- $\ell_{33} = \sqrt{98 - (-8)^2 - 5^2} = \sqrt{9} = 3$.

Resultado:

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix}.$$

Verificación:

$$LL^\top = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix} \begin{bmatrix} 2 & 6 & -8 \\ 0 & 1 & 5 \\ 0 & 0 & 3 \end{bmatrix} = \begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix} = A. \checkmark$$

Implementación en Numpy

Ejemplo de código de cálculo y verificación:

```
import numpy as np

A = np.array([[  4,  12, -16],
              [ 12,  37, -43],
              [-16, -43,  98]], dtype=float)

L = np.linalg.cholesky(A)
print(L)
print(np.allclose(L @ L.T, A))   # True
```

3.4 Cholesky frente a LU

Cuando A es SDP, Cholesky es preferible a LU:

- **La mitad de costo:** $\frac{1}{3}n^3$ operaciones en lugar de $\frac{2}{3}n^3$.
- **La mitad de almacenamiento:** sólo L; la simetría de A evita guardar U.
- **Sin pivoteo:** es numéricamente estable sin necesidad de intercambiar filas o columnas.

3.5 Aplicaciones

La descomposición de Cholesky tiene las mismas aplicaciones que LU — resolución de sistemas y cálculo de determinantes — y algunas específicas de matrices SDP:

- simulación de variables aleatorias correlacionadas (dada una matriz de covarianza Σ, si $z \sim \mathcal{N}(0, I)$ entonces $Lz \sim \mathcal{N}(0, \Sigma)$ con L el factor de Cholesky de Σ);
- problemas de optimización con formas cuadráticas $x^\top Ax$;
- cálculo de distancias de Mahalanobis $d(x, y) = \sqrt{(x-y)^\top \Sigma^{-1}(x-y)}$;
- factorización eficiente del paso de proyección en mínimos cuadrados con regularización.

3.6 Existencia y unicidad

Teorema. Si A es simétrica definida positiva, entonces existe una única descomposición $A = LL^\top$ con L triangular inferior de diagonal estrictamente positiva.

El caso **semi-definido positivo** (SSDP, con algún valor propio nulo) es más delicado. Admitiendo elementos diagonales nulos en L, la descomposición existe pero no es necesariamente única. Por ejemplo, para

$$A = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix},$$

cualquier $L = \begin{bmatrix} 0 & 0 \\ x & y \end{bmatrix}$ con $x^2 + y^2 = 1$ satisface $LL^\top = A$. Si en cambio reordenamos filas y columnas para obtener

$$\tilde{A} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix},$$

la descomposición es única: $\tilde{L} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$.

Para matrices no simétricas el factor $LL^\top$ es siempre simétrico, así que Cholesky no aplica; lo mismo ocurre para matrices no positivas, ya que $LL^\top$ es siempre semidefinida positiva.

3.7 Aspectos numéricos

Cholesky es numéricamente estable cuando A es estrictamente definida positiva. Cerca del borde del cono SDP (es decir, cuando algun valor propio se aproxima a 0) la factorización pierde estabilidad. Consideremos

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Aplicando el algoritmo:

$$\ell_{11} = 1, \quad \ell_{21} = 1, \quad \ell_{22} = \sqrt{1 - 1^2} = 0.$$

Matemáticamente bien, pero si por error de redondeo resulta $\ell_{21} = 1 + \varepsilon$ con $\varepsilon > 0$, entonces $\ell_{22} = \sqrt{-2\varepsilon - \varepsilon^2}$ queda indefinido (raíz de número negativo). Este fenómeno se mitiga con **pivoteo**: intercambiar filas y columnas para llevar el elemento más grande al pivote, obteniendo $P^\top AP = LL^\top$.

Otra consideración numérica: el cálculo de raíces cuadradas es más caro que las operaciones aritméticas básicas. La alternativa es la **descomposición** $LDL^\top$, con L triangular inferior unitaria y D diagonal:

$$A = LDL^\top.$$

Esta forma evita raíces cuadradas para los elementos de la diagonal y se usa internamente en muchas bibliotecas numéricas.

3.8 Ejercicios sugeridos

1. Calcular la descomposición de Cholesky de $A = \begin{bmatrix} 9 & 6 \\ 6 & 5 \end{bmatrix}$ y usarla para resolver $Ax = \begin{bmatrix} 21 \\ 15 \end{bmatrix}$.
2. Verificar que si L es el factor de Cholesky de Σ y $z \sim \mathcal{N}(0, I_n)$, entonces $\mathrm{Cov}(Lz) = \Sigma$.
3. Mostrar que para cualquier matriz L triangular inferior con diagonal no negativa, el producto $LL^\top$ es simétrico semidefinido positivo.

4. Descomposición QR

> ***Lecturas sugeridas:*** *Strang, Introduction to Linear Algebra, sección 4.4.*

4.1 Definición

Para $A \in \mathbb{R}^{m\times n}$ con $m \geq n$ **y rango completo por columnas** ($\text{rango}(A) = n$), la descomposición QR (reducida) expresa

$$A = QR,$$

donde:

- $Q \in \mathbb{R}^{m\times n}$ tiene columnas ortonormales ($Q^\top Q = I_n$, pero en general $QQ^\top \neq I_m$, a menos que $m = n$),
- $R \in \mathbb{R}^{n\times n}$ es triangular superior.

> ***Nota.*** *Si A es de rango completo, el número de columnas de Q es el mismo que A. En general Q tiene tantas columnas como la dimensión del espacio columna de A, es decir $\text{rango}(A) \leq n$. El caso de rango deficiente se trata en la sección 4.4.*

Por ejemplo, para A de 3×2:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} = \begin{bmatrix} q_{11} & q_{12} \\ q_{21} & q_{22} \\ q_{31} & q_{32} \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} \\ 0 & r_{22} \end{bmatrix}.$$

Interpretación geométrica. Las columnas de Q son una base ortonormal del espacio columna de A. La j-ésima columna de R contiene las coordenadas de la j-ésima columna de A en esa base. La elección de Q es precisamente la que hace que R resulte triangular superior: al introducir las columnas de A una por una, cada nueva columna sólo introduce una dirección adicional.

Para matrices "anchas" ($m \leq n$) la noción se extiende de manera natural: Q es $m \times m$ ortogonal y R es $m \times n$ trapezoidal superior. Por ejemplo, para A de 3×4:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \end{bmatrix} = \begin{bmatrix} q_{11} & q_{12} & q_{13} \\ q_{21} & q_{22} & q_{23} \\ q_{31} & q_{32} & q_{33} \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & r_{14} \\ 0 & r_{22} & r_{23} & r_{24} \\ 0 & 0 & r_{33} & r_{34} \end{bmatrix}.$$

4.2 El proceso de Gram-Schmidt

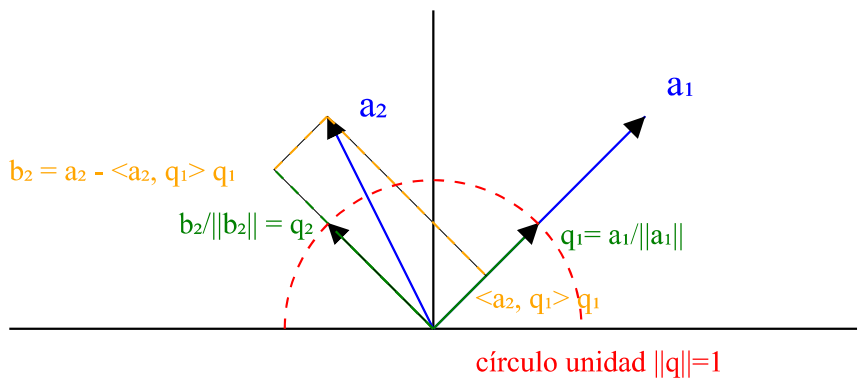
La forma conceptualmente más directa de calcular Q y R es el **proceso de Gram-Schmidt**: construir la base ortonormal columna a columna, restando a cada nueva columna de A su proyección sobre las direcciones anteriores.

Sean $a_1, a_2, \ldots, a_n$ las columnas de A y $q_1, q_2, \ldots, q_n$ las columnas de Q. El algoritmo es:

1. $q_1 = a_1/\|a_1\|$, con $r_{11} = \|a_1\|$.
2. Para $j = 2, 3, \ldots, n$:

- restar a a_j su proyección sobre las direcciones anteriores: $b_j = a_j - \sum_{i=1}^{j-1} (q_i^\top a_j)\, q_i$,
- normalizar: $q_j = b_j / \|b_j\|$,
- guardar las entradas de R: $r_{ij} = q_i^\top a_j$ para $i < j$ y $r_{jj} = \|b_j\|$.

Los pasos $j = 1$ y $j = 2$ se visualizan en el siguiente diagrama (en azul: columnas de A; en verde: columnas de Q; en naranja: proyecciones):



Costo computacional

En el paso j-ésimo se calculan $j - 1$ productos internos de dimensión m (con su correspondiente escalado y resta), más una normalización final. Cada producto interno con actualización cuesta aprox. $4m$ operaciones. Sumando:

$$\sum_{j=1}^{n} 4m(j-1) + O(mn) \approx 2mn^2.$$

Para una matriz cuadrada ($m = n$) esto es $2n^3$, es decir, tres veces más que LU.

Otros procesos para obtener QR

Gram-Schmidt es facilmente interpretable pero **numéricamente inestable**: la ortogonalidad de Q se pierde rápidamente por acumulación de errores de redondeo. Variantes como **Gram-Schmidt modificado** mejoran algo la estabilidad. Los algoritmos estándar en bibliotecas como LAPACK usan **reflexiones de Householder** o **rotaciones de Givens**, que son considerablemente más estables. No veremos estos métodos en el curso. Por más detalles ver Trefethen & Bau, Lectures 7 y 10.

4.3 Ejemplo: matriz 3×2

Consideremos $A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$.

Columna 1. $\|a_1\| = \sqrt{2}$, luego $q_1 = \frac{1}{\sqrt{2}}[1, 1, 0]^\top$ y $r_{11} = \sqrt{2}$.

Columna 2. Proyección de $a_2 = [1, 0, 1]^\top$ sobre q_1:

$$r_{12} = q_1^\top a_2 = \tfrac{1}{\sqrt{2}}(1 + 0 + 0) = \tfrac{1}{\sqrt{2}}.$$

Componente ortogonal:

$$b_2 = a_2 - r_{12}\, q_1 = [1,0,1]^\top - \tfrac{1}{\sqrt{2}} \cdot \tfrac{1}{\sqrt{2}}[1,1,0]^\top = [\tfrac{1}{2}, -\tfrac{1}{2}, 1]^\top.$$

Su norma es $\|b_2\| = \sqrt{\frac{1}{4} + \frac{1}{4} + 1} = \sqrt{\frac{3}{2}} = r_{22}$, y

$$q_2 = \frac{1}{\sqrt{3/2}}\,[\tfrac{1}{2}, -\tfrac{1}{2}, 1]^\top = \tfrac{1}{\sqrt{6}}[1,-1,2]^\top.$$

Resultado final:

$$Q = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{6} \\ 1/\sqrt{2} & -1/\sqrt{6} \\ 0 & 2/\sqrt{6} \end{bmatrix}, \qquad R = \begin{bmatrix} \sqrt{2} & 1/\sqrt{2} \\ 0 & \sqrt{3/2} \end{bmatrix}.$$

Implementación en Numpy

El siguiente código ejemplifica el cálculo de la descomposición QR y su verificación. Numpy usa internamente un método más estable que Gram-Schmidt, pero el resultado es el mismo.

```
import numpy as np

A = np.array([[1, 1],
              [1, 0],
              [0, 1]], dtype=float)

Q, R = np.linalg.qr(A)    # internamente usa Householder
print(Q)
print(R)
print(np.allclose(Q @ R, A))   # True
```

4.4 Existencia y unicidad

Teorema. Si $A \in \mathbb{R}^{m\times n}$ tiene rango completo por columnas ($\mathrm{rango}(A) = n$), la descomposición QR existe y es única si se exige $r_{jj} > 0$ para todo j. La prueba es constructiva: Gram-Schmidt produce la descomposición y la condición sobre los signos de la diagonal de R elimina la ambigüedad de signo en cada q_j.

Si A no tiene rango completo (rango $r < n$), en algún paso del proceso Gram-Schmidt encuentra $b_j = 0$: la j-ésima columna de A es combinación lineal de las anteriores y no aporta una dirección nueva. Si se mantiene el tamaño de Q en $m \times n$, hay que elegir q_j de manera arbitraria en el complemento ortogonal del espacio ya generado (y la correspondiente fila de R queda nula); la factorización existe pero no es única.

La forma natural de lidiar con esto es reducir Q a su dimensión efectiva: conservar sólo las r direcciones verdaderamente independientes, obteniendo

$$Q \in \mathbb{R}^{m\times r}, \qquad R \in \mathbb{R}^{r\times n}.$$

A esta factorización donde Q tiene menos columnas que A se la llama **descomposición QR reducida**, y es única para cualquier matriz A a menos de signos globales sobre cada columna de Q y los elementos correspondientes de R.

Cuando $r = n$ tenemos la situación de la sección 4.1: la dimensión de Q es la misma que la de A.

4.5 Aplicaciones

- **Resolución de sistemas lineales** (en particular, cuadrados bien condicionados).
- **Mínimos cuadrados** $\min_x \|Ax - b\|_2$ con A rectangular (lo veremos más adelante, junto con la SVD y la pseudoinversa).
- **Iteración QR para valores propios** (próxima sección).

4.5.1 Resolución de sistemas lineales

Dado $Ax = b$ con A cuadrada y $A = QR$:

1. calcular $y = Q^\top b$ (producto matriz-vector, $O(n^2)$),
2. resolver $Rx = y$ por sustitución hacia atrás ($O(n^2)$).

El costo total está dominado por la factorización QR. Respecto a LU, QR es más caro ($2n^3$ frente a $\frac{2}{3}n^3$) pero numéricamente más estable: no requiere pivoteo para matrices bien condicionadas.

4.5.2 Iteración QR para valores propios

La descomposición QR puede utilizarse para calcular valores propios de una matriz simétrica A (y con modificaciones para otras matrices). La idea es iterar:

- descomponer: $A_k = Q_k R_k$,
- recombinar invertido: $A_{k+1} = R_k Q_k$,
- repetir hasta convergencia ($A_{k+1} \approx A_k$).

Cada iteración es una transformación de similitud, es decir, un cambio de base sobre la matriz del paso anterior:

$$A_{k+1} = R_k Q_k = Q_k^\top (Q_k R_k) Q_k = Q_k^\top A_k Q_k,$$

por lo que A_{k+1} y A_k tienen los mismos valores propios. La iteración converge si y solo si $Q_k \to I$, y por tanto $A_k \to R_k$. Es decir, A_k, si converge, converge a una matriz triangular. Además, la matriz debe ser simétrica (por construcción) y cualquier matriz simétrica y triangular es diagonal. Consecuentemente sus elementos diagonales son los valores propios de A.

La demostración y condiciones de convergencia de la iteración QR no es inmediata y no la veremos; con modificaciones, la idea central de este algoritmo es el que usan las bibliotecas numéricas estándar. Por más detalles se recomienda ver Golub & Van Loan, cap. 7; Trefethen & Bau, Lectures 28 y 29.

Ejemplo. Sea $A = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}$, con valores propios $\lambda = 1 \pm \sqrt{2} \approx \{2.414,\ -0.414\}$.

Iteración 0: $A_0 = A$. Su QR (con convención de diagonal positiva en R, como da Gram-Schmidt):

$$Q_0 = \begin{bmatrix} 0.447 & 0.894 \\ 0.894 & -0.447 \end{bmatrix}, \qquad R_0 = \begin{bmatrix} 2.236 & 1.341 \\ 0 & 0.447 \end{bmatrix}, \qquad A_1 = R_0 Q_0 = \begin{bmatrix} 2.20 & 1.40 \\ 0.40 & -0.20 \end{bmatrix}.$$

Iteración 1:

$$Q_1 = \begin{bmatrix} 0.984 & 0.179 \\ 0.179 & -0.984 \end{bmatrix}, \qquad R_1 = \begin{bmatrix} 2.236 & 1.342 \\ 0 & 0.447 \end{bmatrix}, \qquad A_2 = \begin{bmatrix} 2.44 & -0.92 \\ 0.08 & -0.44 \end{bmatrix}.$$

Iteración 2:

$$Q_2 = \begin{bmatrix} 0.999 & 0.033 \\ 0.033 & -0.999 \end{bmatrix}, \qquad R_2 = \begin{bmatrix} 2.441 & -0.934 \\ 0 & 0.410 \end{bmatrix}, \qquad A_3 = \begin{bmatrix} 2.407 & 1.014 \\ 0.013 & -0.410 \end{bmatrix}.$$

La diagonal de A_k se acerca ya a los valores propios $\{2.414, -0.414\}$, y el elemento subdiagonal tiende a cero.

4.6 Ejercicios sugeridos

1. Calcular la descomposición QR de $A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$ por Gram-Schmidt.
2. Usar la QR del ejercicio anterior para resolver $\min_x \|Ax - b\|_2$ con $b = [1, 1, 1]^\top$ (pista: $x = R^{-1}Q^\top b$ cuando R es cuadrada y no singular).
3. Iterar el algoritmo QR dos pasos más sobre la matriz $A = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}$ del ejemplo anterior y verificar numéricamente la convergencia hacia los valores propios teóricos.